PRACTICAL - 11

Implementation of Floyd-Warshall Algorithm

Aim of the Practical:

To implement the Floyd-Warshall algorithm for all-pairs shortest paths and derive its cubic time complexity.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    const int INF = 1000000;
    vector<vector<int>> d = {
        {0,5,INF,10},
        {INF,0,3,INF},
        {INF,INF,0,1},
        {INF,INF,INF,0}
    };
    int V = d.size();
    for (int k = 0; k < V; k++)
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    for (auto &row : d) {
        for (int x : row)
            cout << (x >= INF ? -1 : x) << "\t";
        cout << endl;
    }
}
```

Output:

```
0       5       8       9
-1      0       3       4
-1      -1      0       1
-1      -1      -1      0
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices.

The algorithm has three nested loops over k, i and j.

k-loop iterations = V

i-loop iterations for each k = V

j-loop iterations for each (k,i) = V

Inside the innermost loop, only constant-time arithmetic, comparison and assignment are performed.

$T(V) = V \cdot V \cdot V \cdot \Theta(1)$

$T(V) = \Theta(V^3)$

The distance table contains V^2 entries.

$S(V) = \Theta(V^2)$

Final Time Analysis:

- Time Complexity: $\Theta(V^3)$.
- Space Complexity: $\Theta(V^2)$.

Name: B.SIDHARTHA **Enrollment No: 92460118150**

Result:

Floyd-Warshall successfully computed the shortest distance between every ordered pair of vertices.